Alcances (Scopes)

En Node-RED, puedes tener variables en distintos alcances:

Tipo de variable	¿Dónde vive?	¿Quién la puede ver?
`var`, `let`, `const`	Dentro del nodo `function`	Solo ese nodo
`flow.set()` / `flow.get()`	En el flujo actual	Todos los nodos del mismo flujo
`global.set()` / `global.get()`	En todo Node-RED	Todos los nodos, todos los flujos

Diferencias entre `var`, `let` y `const` en JavaScript

Característica	`var`	`let`	`const`
Ámbito (scope)	Función	Bloque (`{}`)	Bloque (`{}`)
Re-declarable	Sí	No	No
Re-asignable	Sí	Sí	No
Hoisting	Sí (sube sin valor)	Parcial (no inicializado)	Parcial (no inicializado)
Uso recomendado	Evitar en código moderno	Para variables cambiantes	Para constantes y objetos

Ejemplo

```
function ejemplo() {
  if (true) {
    var x = 1;
    let y = 2;
    const z = 3;
  }
  console.log(x); //  1
  console.log(y); //  Error: y no está definido
  console.log(z); //  Error: z no está definido
}
```

set y get en Node-red

Método	¿Qué hace?	Ejemplo
`global.set("x", 5)`	Guarda el valor `5` en `x` global	Disponible en todos los flujos
`global.get("x")`	Recupera el valor de `x`	Devuelve `5` si existe
`flow.set("x", 5)`	Guarda el valor en el flujo actual	Solo accesible en ese flujo
`flow.get("x")`	Recupera el valor en ese flujo	Devuelve `5` si existe

Tipos de dato

JavaScript tiene **tipos primitivos** y **estructurados**:

Primitivos

Tipo	Ejemplo
`string`	`"Hola"`
`number`	`42`, `3.14`
`boolean`	`true`, `false`
`undefined`	`let x;`
`null`	`let x = null;`
`bigint`	`12345678901234567890n`
`symbol`	`Symbol("id")`

Estructurados

Tipo	Ejemplo
`object`	`{a: 1, b: 2}`
`array`	`[1, 2, 3]`
`function`	`function() {...}`
`Date`	`new Date()`
`RegExp`	`/abc/`

Node-red

`Buffer`	Archivos binarios, imágenes, audio

Conversión de tipos (casting)

A `string`

```
let texto = String(42);        // "42"
let texto2 = JSON.stringify(obj); // convierte objeto a texto JSON
```

A `number`

```
let num = Number("3.14");      // 3.14
let entero = parseInt("42");   // 42
```

A `boolean`

```
let estado = Boolean("texto");  // true
let estado2 = !!variable;       // doble negación
```

A `object` (desde JSON)

```
let obj = JSON.parse('{"a":1}'); // convierte texto JSON a objeto
```

A `array`

```
let lista = Array.from("abc");  // ["a", "b", "c"]
```